(4) Progress Report-3

(20-Feb-2026)

Agile Documentation

(1) Agile Project Charter

Project Name: CricVerse **Vision**: To create a comprehensive, real-time cricket scoring and management application with integrated e-commerce capabilities. **Mission**: Deliver an intuitive web platform for cricket enthusiasts, tournament organizers, and players to seamlessly manage matches, track detailed statistics, and purchase cricket merchandise. **Success Criteria**:

- Successful robust implementation of real-time ball-by-ball scoring.
- Intuitive team, player, and tournament management.
- Fully functional e-commerce store with secure shopping cart management. **Primary Stakeholders**: End-users (scorers, players, fans), Administrators/Organizers, Customers, Development Team.

(2) Agile Roadmap / Schedule

- **Phase 1: Foundation & Team Management** (Models, UI, Basic API) - completed
- **Phase 2: Core Match Engine** (Live Scoring, Innings management) - in progress
- **Phase 3: Tournaments & Leaderboards** (Fixtures, Standings) - upcoming
- **Phase 4: E-commerce & Polish** (Store, Cart, Notifications) - upcoming
- **Phase 5: Final Review & Production Deployment** - future

(3) Agile Project Plan

Methodology: Scrum Framework with iterative development sprints. **Current Phase Status**: Phase 2 (Match Engine Development). **Dependencies**: MongoDB connection stability, Next.js App Router API performance, and React Context optimizations. **Risk Management**:

- *Risk*: Data inconsistency during rapid real-time scoring.
- *Mitigation*: Utilize optimized React Context state management combined with optimistic UI updates and strict API validation.

(4) Agile User Story (Current Focus)

- **Epic**: Real-time cricket scoring console.
- **Story 1**: As a team manager, I want to create a team and add players so that we can participate in matches (Completed).
- **Story 2**: As a scorer, I want to configure the match settings (overs, pitch, toss) so that the match is recorded correctly (Completed).
- **Story 3**: As a scorer, I want to record runs, wickets, and extras ball-by-ball using keyboard shortuts so that the match scorecard is updated rapidly (In Progress).
- **Story 4**: As a user, I want to view live run-rate graphs to visually analyze the team's performance (In Progress).

(5) Agile Release Plan

- **Release 0.5 (Alpha)**: Core schemas, team creation, player management, static UI elements. (Completed)
- **Release 1.0 (Beta)**: Fully playable single matches, live scoring engine, toss configuration. (Target: End of March 2026)

- **Release 2.0 (Production)**: Tournaments, Leaderboards, Store integration, Admin dashboard. (Target: August 2026)

(6) Agile Sprint Backlog

Current Sprint Focus: Match Engine API & Live Scoring UI

- [x] Design MongoDB schemas for `Match`, `Innings`, and `Ball`.
- [x] Implement frontend UI for Match configuration (Toss, Overs, Pitch).
- [ ] Create Next.js API endpoints for POST/GET matches (`app/api/matches`).
- [ ] Integrate live scoring logic (runs, wickets, extras) with React State.
- [ ] Build Recharts component for real-time run-rate visualization.
- [ ] Ensure ball-by-ball data flows properly to MongoDB.

(7) Agile Test Plan

Unit Testing:

- Validate stats calculations (Average, Strike Rate, Economy for `BattingStats` and `BowlingStats`).
- Test Next.js API route handlers using mocking (`app/api/teams` and `app/api/matches`). **Integration Testing**:
- Verify data flow from Match creation form to the MongoDB database.
- Test Shopping Cart context state consistency. **User Acceptance Testing (UAT)**:
- Conduct simulated matches with designated scorers to validate the ease and speed of the live scoring UI keyboard shortcuts.
- Verify that team captains can successfully update their player rosters.